Pokédex con Ionic

Práctica guiada en 2 etapas

Andrick Millan y Emiliano Cisneros

25 de octubre de 2026

1 Qué vamos a construir

Una app de Pokémon con dos pantallas: una **lista** con imágenes y, al tocar una tarjeta, una pantalla de **detalle** con tipos, habilidades, altura y peso que se piden a internet (PokeAPI).

Etapa	Qué se logra	Concepto nuevo	Min
0	Crear el proyecto y verlo en el navegador	`ionic start`, `ionic serve`	10
1	Lista de 3 Pokémon con imagen	Componentes de Ionic, `@for`	20
2	Pantalla de detalle con datos de internet	Rutas, `HttpClient`, signals	30

Cómo usar esta guía

En cada etapa: abre o crea los archivos indicados, **reemplaza todo su contenido** con el código de la guía, guarda con `Ctrl+S` y mira el navegador (se actualiza solo).

Reglas para evitar errores

- Crea los archivos **a mano** (clic derecho → New File). No uses `ionic generate`.
- Los nombres de archivo deben ser **exactamente** los de la guía (minúsculas, con punto).
- Crea el proyecto en una ruta corta, sin acentos ni espacios: `C:\Proyectos` (Mac: `~/Proyectos`).
- Comentarios: en `.ts` se escribe `// texto`; en `.html` se escribe `<!-- texto -->`. Si pones `//` en un HTML, el texto aparece en la pantalla.
- Las comillas especiales del código (`) se llaman *comillas invertidas*. Windows (teclado español): tecla ` y luego espacio, o `Alt Gr + }`. Mac: `Option + ` y espacio.

2 Etapa 0: preparar el proyecto (10 min)

Requisitos previos

- Node.js **22.22.3 o superior** (recomendado la LTS más reciente). Comprobar: `node -v`
- Ionic CLI: `npm install -g @ionic/cli`. Comprobar: `ionic --version`
- Visual Studio Code.

Pasos

1. En VS Code abre una terminal (`Ctrl+‘`) y entra a tu carpeta de trabajo:

```
cd C:\Proyectos          # Mac: cd ~/Proyectos
```

2. Crea el proyecto (si pregunta por una cuenta de Ionic, responde **n**):

```
ionic start pokedex blank --type=angular-standalone --capacitor
cd pokedex
```

3. Abre la carpeta del proyecto en VS Code: `File` → `Open Folder` → `pokedex`.
4. Arranca la app:

```
ionic serve
```

Resultado esperado

Se abre `http://localhost:8100` con la página de bienvenida. **No cierres la terminal**: mientras `ionic serve` corra, cada cambio guardado se ve solo. Para vista de celular: `F12` y el icono de teléfono.

Verifica esto

Abre `src/app/home/home.page.ts`. Los componentes de Ionic deben importarse de `’@ionic/angular’`. Si dice `’@ionic/angular/standalone’`, cámbialo por `’@ionic/angular’` en todos los códigos de esta guía.

3 Etapa 1: lista de Pokémon (20 min)

Objetivo: mostrar 3 Pokémon escritos a mano para entender cómo se arma una pantalla con Ionic.

Pasos

Reemplaza el contenido de los tres archivos de `src/app/home/`.

Archivo: `src/app/home/home.page.ts`

```
import { Component } from '@angular/core';
import {
  IonHeader,
  IonToolbar,
  IonTitle,
  IonContent,
  IonCard,
  IonCardHeader,
  IonCardTitle,
  IonCardSubtitle,
```

```
} from '@ionic/angular';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  imports: [
    IonHeader,
    IonToolbar,
    IonTitle,
    IonContent,
    IonCard,
    IonCardHeader,
    IonCardTitle,
    IonCardSubtitle,
  ],
})
export class HomePage {
  // Lista escrita a mano (todavia no usamos internet)
  pokemons = [
    { id: 1, nombre: 'bulbasaur' },
    { id: 4, nombre: 'charmander' },
    { id: 25, nombre: 'pikachu' },
  ];

  // Devuelve la URL de la imagen de un Pokemon segun su numero
  imagen(id: number): string {
    return `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/
    official-artwork/${id}.png`;
  }
}
```

Archivo: src/app/home/home.page.html

```
<ion-header>
  <ion-toolbar color="danger">
    <ion-title>Pokédex</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  @for (p of pokemons; track p.id) {
    <ion-card>
      <img [src]="imagen(p.id)" [alt]="p.nombre" />
      <ion-card-header>
        <ion-card-subtitle>#{{ p.id }}</ion-card-subtitle>
        <ion-card-title>{{ p.nombre }}</ion-card-title>
      </ion-card-header>
    </ion-card>
  }
</ion-content>
```

Archivo: src/app/home/home.page.scss

```
/* Las imagenes ocupan todo el ancho de la tarjeta */
img {
  width: 100%;
  height: auto;
  display: block;
}
```

Resultado esperado

Tres tarjetas con la imagen de Bulbasaur, Charmander y Pikachu y una barra roja con el título *Pokédex*.

Qué aprendimos

- Una **página** son tres archivos: `.ts` (lógica), `.html` (lo que se ve) y `.scss` (estilo).
- `ion-header`, `ion-toolbar`, `ion-content`, `ion-card`: componentes ya hechos de Ionic. Cada uno que se usa en el HTML debe estar en la lista `imports` del componente **y** en el `import` de arriba.
- `@for` repite un bloque por cada elemento de la lista; `track` indica qué dato identifica a cada elemento.
- `[src]="..."` conecta un valor del código con el HTML; `{{ ... }}` imprime un valor.
- Cada número es el de la Pokédex real: 1 Bulbasaur, 4 Charmander, 25 Pikachu.

Errores comunes

Un componente sale en gris en la lista de imports	Está importado arriba pero falta en `imports: [...]`.
`'ion-card' is not a known element`	Agrega `IonCard` a `imports: [...]`.
Imágenes rotas	La URL debe ir entre **comillas invertidas**, sin espacios, y terminar en `${id}.png`.
Pantalla en blanco	Revisa la terminal y la consola (`F12`); suele ser una llave sin cerrar.

4 Etapa 2: pantalla de detalle (30 min)

Objetivo: al tocar una tarjeta, abrir otra pantalla con los datos de ese Pokémon, pedidos a la API.

Idea

Una **API** es un servicio al que le pedimos datos por internet. Abre `https://pokeapi.co/api/v2/pokemon/25` en el navegador: es un JSON con todo lo de Pikachu (`types`, `abilities`, `height`, `weight`). Nuestra pantalla de detalle pide ese JSON y muestra las partes que nos interesan.

Pasos

1. **Permitir peticiones a internet.** En `src/main.ts` agrega el `import` y la línea `provideHttpClient()` dentro de `providers`. Puedes reemplazar todo el archivo:

Archivo: `src/main.ts`

```
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withComponentInputBinding, withPreloading,
    PreloadAllModules } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
```

```
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules), withComponentInputBinding()),
    provideHttpClient(), // <- permite consultar la PokeAPI
  ],
});
```

2. **Agregar la ruta** de la pantalla nueva (`detalle/:id`; los dos puntos indican que el número cambia). Reemplaza `src/app/app.routes.ts`:

Archivo: `src/app/app.routes.ts`

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'home',
    loadComponent: () => import('./home/home.page').then((m) => m.HomePage),
  },
  {
    path: 'detalle/:id',
    loadComponent: () => import('./detalle/detalle.page').then((m) => m.DetallePage),
  },
  {
    path: '',
    redirectTo: 'home',
    pathMatch: 'full',
  },
];
```

3. **Hacer que la tarjeta sea un enlace.** En `home.page.ts` se agrega `RouterLink` (en el `import` de `@angular/router` **y** dentro de `imports: [...]`; no va en el bloque de Ionic). En `home.page.html` la tarjeta lleva `[routerLink]`. Reemplaza ambos archivos:

Archivo: `src/app/home/home.page.ts`

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
import {
  IonHeader,
  IonToolbar,
  IonTitle,
  IonContent,
  IonCard,
  IonCardHeader,
  IonCardTitle,
  IonCardSubtitle,
} from '@ionic/angular';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  imports: [
    RouterLink,
```

```
    IonHeader,
    IonToolbar,
    IonTitle,
    IonContent,
    IonCard,
    IonCardHeader,
    IonCardTitle,
    IonCardSubtitle,
  ],
})
export class HomePage {
  pokemons = [
    { id: 1, nombre: 'bulbasaur' },
    { id: 4, nombre: 'charmander' },
    { id: 25, nombre: 'pikachu' },
  ];

  imagen(id: number): string {
    return `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/
    official-artwork/${id}.png`;
  }
}
```

Archivo: `src/app/home/home.page.html`

```
<ion-header>
  <ion-toolbar color="danger">
    <ion-title>Pokédex</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  @for (p of pokemons; track p.id) {
    <ion-card [routerLink]="['/detalle', p.id]">
      <img [src]="imagen(p.id)" [alt]="p.nombre" />
      <ion-card-header>
        <ion-card-subtitle>#{{ p.id }}</ion-card-subtitle>
        <ion-card-title>{{ p.nombre }}</ion-card-title>
      </ion-card-header>
    </ion-card>
  }
</ion-content>
```

4. **Crear la pantalla de detalle.** Crea la carpeta `src/app/detalle/` (clic derecho en `app` → New Folder) y dentro dos archivos nuevos:

Archivo: `src/app/detalle/detalle.page.ts`

```
import { Component, inject, signal } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { HttpClient } from '@angular/common/http';
import {
  IonHeader,
  IonToolbar,
  IonButtons,
  IonBackButton,
  IonTitle,
  IonContent,
  IonChip,
  IonSpinner,
} from '@ionic/angular';

@Component({
```

```
  selector: 'app-detalle',
  templateUrl: 'detalle.page.html',
  imports: [
    IonHeader,
    IonToolbar,
    IonButtons,
    IonBackButton,
    IonTitle,
    IonContent,
    IonChip,
    IonSpinner,
  ],
})
export class DetallePage {
  private ruta = inject(ActivatedRoute);
  private http = inject(HttpClient);

  // Aquí guardamos los datos que llegan de internet (al inicio no hay nada)
  pokemon = signal<any>(null);

  constructor() {
    // Leemos el número que viene en la dirección: /detalle/25
    const id = this.ruta.snapshot.paramMap.get('id');

    // Pedimos los datos de ese Pokémon a la PokéAPI
    this.http
      .get<any>(`https://pokeapi.co/api/v2/pokemon/${id}`)
      .subscribe((datos) => this.pokemon.set(datos));
  }

  imagen(id: number): string {
    return `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/
    official-artwork/${id}.png`;
  }
}
```

Archivo: src/app/detalle/detalle.page.html

```
<ion-header>
  <ion-toolbar color="danger">
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>Detalle</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  @if (pokemon(); as p) {
    <div class="ion-text-center">
      <img [src]="imagen(p.id)" [alt]="p.name" style="width: 220px; max-width: 70%" />
      <h1>{{ p.name }}</h1>
    </div>

    <h3>Tipos</h3>
    @for (t of p.types; track t.type.name) {
      <ion-chip>{{ t.type.name }}</ion-chip>
    }

    <h3>Habilidades</h3>
    @for (h of p.abilities; track h.ability.name) {
      <ion-chip color="danger">{{ h.ability.name }}</ion-chip>
    }
```

```
    <h3>Medidas</h3>
    <p>Altura: {{ p.height / 10 }} m</p>
    <p>Peso: {{ p.weight / 10 }} kg</p>
  } @else {
    <div class="ion-text-center">
      <ion-spinner></ion-spinner>
    </div>
  }
</ion-content>
```

5. Guarda todo (`File` → `Save All`). Si la terminal muestra un error viejo, deten el servidor con `Ctrl+C` y ejecuta de nuevo `ionic serve`.

Resultado esperado

Al tocar la tarjeta de Pikachu, la URL cambia a `/detalle/25` y se ve el Pokémon grande con sus tipos (*electric*), sus habilidades (*static*, *lightning-rod*), altura 0.4 m y peso 6 kg. La flecha de regreso vuelve a la lista.

Qué aprendimos

- **Ruta con parámetro:** `detalle/:id`; el número se lee con `ActivatedRoute`.
- `routerLink`: navegar sin recargar la página.
- `HttpClient`: pedir datos a una API.
- **signal:** variable "reactiva". Se lee con paréntesis, `pokemon()`, y se cambia con `.set(...)`; cuando cambia, la pantalla se actualiza sola.
- `@if / @else`: mostrar cosas según una condición (aquí, el *spinner* mientras llegan los datos).

Errores comunes

`Could not resolve ./detalle/detalle.page`	El archivo `detalle.page.ts` no existe, está vacío o con otro nombre. Si ya está bien, reinicia `ionic serve`.
`Can't bind to 'routerLink'`	Falta `RouterLink` en `imports: [...]` de `home.page.ts`.
`No provider for HttpClient`	Falta `provideHttpClient()` en `main.ts`.
Sale texto raro con `//` en la pantalla	Un comentario de HTML debe ser `<!-- ... -->`.
Spinner infinito	Sin internet o URL mal escrita. Revisa la consola (`F12`).
Ves `RouterLink` en rojo	Está dentro del bloque `from '@ionic/angular'`; debe ir en su propio `import` de `'@angular/router'`.

5 Cierre y retos

- **Reto 1:** agrega otro Pokémon a la lista (por ejemplo, 7 Squirtle o 150 Mewtwo).
- **Reto 2:** en el detalle, muestra también el `base_experience` del Pokémon (está en el JSON).
- **Reto 3:** cambia el color de la barra superior (`color="danger"`) por `primary`, `success` o `warning`.

- (Opcional, Android) `ionic build`, `npx cap add android`, `npx cap open android`: la misma app corre como app nativa.

6 Solución de problemas generales

Problema	Solución
La terminal no reconoce `ionic`	Cierra y vuelve a abrir VS Code; si sigue, reinstala con `npm install -g @ionic/cli`.
PowerShell bloquea scripts	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Errores raros de compilación en Windows	Crea el proyecto en `C:\Proyectos`, sin acentos ni carpetas redirigidas.
Puerto 8100 ocupado	`ionic serve --port=8101`
Guardé y no cambia nada	Revisa que `ionic serve` siga corriendo; si hay un error viejo, `Ctrl+C` y vuelve a ejecutarlo.
Formatear el código en VS Code	`Shift+Alt+F` (Mac: `Shift+Option+F`).